

Part 1- Code Review & Debugging

```
@app.route('/api/products', methods=['POST'])  
  
def create_product():  
    data = request.json
```

Problem 1 : There should be validation of the received data

Solution :

Validation should include-

- i. Check if all fields are filled or not, if not throw error (using status codes) (Atleast for important fields like- name , sku)
- ii. Make sure price of item should not be negative i.e. price > 0 always
- iii. Type cast price variable into decimal (as mentioned).
- iv. Using try catch block (exception handling will be better)

Correct code :

#validate req fields

```
required_fields = ['name', 'sku']  
for field in required_fields:  
    if field not in data:  
        return {"error": f"{field} is required"}, 400
```

#optional fields

```
price = Decimal(str(data.get('price', 0)))  
initial_quantity = int(data.get('initial_quantity', 0))  
warehouse_id = data.get('warehouse_id')
```

#validate input

```
if price < 0:  
    return {"error": "Price cannot be negative"}, 400
```

```
if initial_quantity < 0:  
    return {"error": "Quantity cannot be negative"}, 400
```

Impact in production:

API becomes unreliable (500 errors)

Bad data corrupts your system

```
# Create new product
product = Product(
    name=data['name'],
    sku=data['sku'],
    price=data['price'],
    warehouse_id=data['warehouse_id']
)
```

Problem 2 : Sku should be unique, but there is no check.

Solution : check if it is already present , if present throw error

Correct code :

```
# 2. Check SKU uniqueness

existing = Product.query.filter_by(sku=data['sku']).first()

if existing:

    return {"error": "SKU already exists"}, 409
```

```
db.session.add(product)
db.session.commit()

# Update inventory count
inventory = Inventory(
    product_id=product.id,
    warehouse_id=data['warehouse_id'],
    quantity=data['initial_quantity']
)
```

Problem 3 : Incorrect Business Logic (Multi-Warehouse)

Solution - Validate if warehouse id exist, if yes validate it with actual , if not skip it(because it might not be provided)

```
# 3. Validate warehouse----
warehouse = None
if warehouse_id:
    warehouse = Warehouse.query.get(warehouse_id)
if not warehouse:
    return {"error": "Invalid warehouse_id"}, 400
```

Impact-

Tightly couples product to one warehouse

Breaks scalability of system design

```
db.session.add(inventory)

db.session.commit()

return {"message": "Product created", "product_id": product.id}
```

Problem 3 :Two Separate DB Commits (No Transaction Safety)

```
i.e- db.session.commit();

.....

db.session.commit();
```

Impact:

Product created but inventory fails- inconsistent data

You now have products with no inventory

Correct code –

```
# 4. Use transaction
with db.session.begin():
    product = Product(
        name=data['name'],
        sku=data['sku'],
        price=price
    )
    db.session.add(product)
    db.session.flush() # get product.id without commit
```

Create inventory ONLY if warehouse provided

```
if warehouse_id:
    inventory = Inventory(
```

```
product_id=product.id,  
warehouse_id=warehouse_id,  
quantity=initial_quantity  
)  
db.session.add(inventory)
```

```
return {  
    "message": "Product created",  
    "product": {  
        "id": product.id,  
        "name": product.name,  
        "sku": product.sku,  
        "price": str(product.price)  
    }  
}, 201
```

```
except IntegrityError:  
    db.session.rollback()  
    return {"error": "Database integrity error"}, 500
```

```
except Exception as e:  
    db.session.rollback()  
    return {"error": str(e)}, 500
```

Final List of Errors-

1. No Input Validation
2. SKU Uniqueness Not Enforced
3. Two Separate DB Commits (No Transaction Safety)
4. Incorrect Business Logic (Multi-Warehouse)
5. No Error Handling- No try/except

Impact:

Any DB failure → server crash / 500

No rollback → partial writes

6. Decimal Handling for Price

```
price=data['price']
```

Impact:

Float precision errors (VERY BAD for money)

Financial inaccuracies

7. Inventory Creation Assumes initial_quantity Exists

quantity=data['initial_quantity']

Impact:

KeyError if missing

Business logic unclear (should default to 0?)

8. No Validation for Warehouse Existence

Impact :

Foreign key errors OR silent data inconsistency

9. No Idempotency I.e Duplicate Requests Handling

If client retries request - duplicate product

Impact:

Double entries from retries or network issues

10. Response Too Minimal

return {"message": "Product created", "product_id": product.id}

Impact:

Poor DX (developer experience)

Frontend may need more data

Part – 2 – Database Design

1. Companies Table

Reason: Multiple companies use the SaaS platform

```
companies (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
)
```

2. Warehouses Table

Reason: A company can have multiple warehouses

```
warehouses (  
  id SERIAL PRIMARY KEY,  
  company_id INT REFERENCES companies(id),  
  name VARCHAR(255),  
  location VARCHAR(255),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
)
```

3. Products Table

Reason: Stores product details (independent of warehouse)

```
products (  
  id SERIAL PRIMARY KEY,  
  company_id INT REFERENCES companies(id),  
  name VARCHAR(255) NOT NULL,  
  sku VARCHAR(100) UNIQUE NOT NULL,  
  price DECIMAL(10,2),  
  is_bundle BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
)
```

4. Inventory Table

Reason: Handles many-to-many between products and warehouses

```
inventory (  
  id SERIAL PRIMARY KEY,  
  product_id INT REFERENCES products(id),  
  warehouse_id INT REFERENCES warehouses(id),  
  quantity INT DEFAULT 0,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  UNIQUE(product_id, warehouse_id)  
)
```

5. Inventory Logs Table

Reason: Track changes in inventory (IMPORTANT requirement)

```
inventory_logs (  
  id SERIAL PRIMARY KEY,  
  product_id INT,  
  warehouse_id INT,  
  change_type VARCHAR(50),-- ADD / REMOVE / ADJUST  
  quantity_changed INT,  
  previous_quantity INT,  
  new_quantity INT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
)
```

6. Suppliers Table

Reason: Suppliers provide products

```
suppliers (  
  id SERIAL PRIMARY KEY,  
  company_id INT REFERENCES companies(id),  
  name VARCHAR(255),  
  contact_info TEXT  
)
```

7. Product-Supplier Mapping

Reason: Many suppliers can provide same product

```
product_suppliers (  
  id SERIAL PRIMARY KEY,  
  product_id INT REFERENCES products(id),  
  supplier_id INT REFERENCES suppliers(id),  
  cost_price DECIMAL(10,2)  
)
```

8. Bundle Items Table

Reason: Products can be bundles of other products

```
bundle_items (  
  id SERIAL PRIMARY KEY,  
  bundle_product_id INT REFERENCES products(id),  
  component_product_id INT REFERENCES products(id),  
  quantity INT  
)
```

Design Decisions

Decision 1: Separate Inventory Table

Problem:

Product can exist in multiple warehouses

Solution:

Use an inventory table instead of storing warehouse_id in product

Impact:

- Supports multi-warehouse system

- Scalable and flexible

Decision 2: Unique Constraint on SKU

```
sku VARCHAR(100) UNIQUE
```

Reason:

SKU must uniquely identify product across platform

Impact:

- Prevents duplicate products

- Easier integrations

Decision 3: Composite Unique Index

```
UNIQUE(product_id, warehouse_id)
```

Reason:

A product should have only ONE inventory record per warehouse

Impact:

Prevents duplicate inventory rows

Decision 4: Inventory Logs Table

Reason:

Requirement: "Track when inventory levels change"

Impact:

Audit trail

Debugging

Analytics

Decision 5: Bundle Design (Self-referencing)

Reason:

A product can contain other products

Impact:

Flexible bundle system

Supports kits and combos

Decision 6: Decimal for Price

price DECIMAL(10,2)

Reason:

Avoid float precision issues

Impact:

Ensures financial accuracy

Questions to ask product team

1. Can one product belong to multiple companies, or is it strictly scoped to one company?
2. Should SKU be unique globally across the platform or only within a company?
3. Can a product exist without being assigned to any warehouse initially?
4. Do we need to support batch tracking, expiry dates, or lot numbers (for domains like pharma/food)?
5. Are suppliers shared across companies or specific to each company?

Part 3 – Api Implementation

Assumptions

1. Each product has a low_stock_threshold field
2. Recent sales = sales in last 30 days
3. Sales data stored in order_items table
4. inventory stores stock per warehouse
5. Each product has at least one supplier (taking first for simplicity)

Implementation (Flask)

```
from datetime import datetime, timedelta

from flask import jsonify

@app.route('/api/companies/<int:company_id>/alerts/low-stock',
methods=['GET'])

def get_low_stock_alerts(company_id):

    try:

        alerts = []

        # define recent sales window

        last_30_days = datetime.utcnow() - timedelta(days=30)

        # get all products for company

        products = Product.query.filter_by(company_id=company_id).all()

        for product in products:

            # check recent sales

            recent_sales = db.session.query(OrderItem).join(Order).filter(

                Order.company_id == company_id,

                OrderItem.product_id == product.id,

                Order.created_at >= last_30_days

            ).count()

            if recent_sales == 0:

                continue # skip products with no recent sales
```

```

# get inventory across warehouses

inventories = Inventory.query.filter_by(product_id=product.id).all()

for inv in inventories:

    if inv.quantity is None:

        continue

    threshold = product.low_stock_threshold or 0

    if inv.quantity < threshold:

        # calculate avg daily sales (simple logic)

        avg_daily_sales = recent_sales / 30 if recent_sales else 0

        days_until_stockout = int(inv.quantity / avg_daily_sales) if
avg_daily_sales > 0 else None

        # get warehouse

        warehouse = Warehouse.query.get(inv.warehouse_id)

        # get supplier (first one)

        supplier = db.session.query(Supplier).join(ProductSupplier).filter(
            ProductSupplier.product_id == product.id
        ).first()

        alerts.append({
            "product_id": product.id,
            "product_name": product.name,
            "sku": product.sku,
            "warehouse_id": inv.warehouse_id,
            "warehouse_name": warehouse.name if warehouse else None,
            "current_stock": inv.quantity,
            "threshold": threshold,
            "days_until_stockout": days_until_stockout,
            "supplier": {
                "id": supplier.id if supplier else None,
                "name": supplier.name if supplier else None,
            }
        })

```

```

        "contact_email": supplier.contact_email if supplier else None
    }
})

return jsonify({
    "alerts": alerts,
    "total_alerts": len(alerts)
}), 200

except Exception as e:
    return {"error": str(e)}, 500

```

Edge Cases Handled

1. No recent sales → skipped
2. No supplier → return null values
3. No warehouse → handled safely
4. Division by zero (sales = 0) → handled
5. Missing threshold → defaults to 0
6. Null quantity → skipped
7. Approach Explanation
8. Filter products by company
9. Check recent sales (last 30 days)
10. For each product → check inventory per warehouse
11. Compare stock with threshold
12. Calculate estimated stockout days
13. Attach supplier info
14. Return aggregated alerts